Karunya INSTITUTE OF TECHNOLOGY AND SCIENCES

(Declared as Deemed to be University under Sec.3 of the UGC Act, 1956)

MoE, UGC & AICTE Approved

NAAC A++ Accredited

DIVISION OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

LABORATORY RECORD

Academic Year 2025-2026

EVEN SEMESTER

Name : **EVANGELINE ARUNA M**

Register Number : **URK23CS5003**

Course Code : **23AI2003**

Course Name : **Artificial Intelligence in Web Development Lab**

APRIL 2026

Karunya INSTITUTE OF TECHNOLOGY AND SCIENCES

(Declared as Deemed to be University under Sec.3 of the UGC Act, 1956)

MoE, UGC & AICTE Approved

NAAC A++ Accredited

DIVISION OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

LABORATORY RECORD

Academic Year 2025-2026

Course Code: 23AI2003

Course Name: Artificial Intelligence in Web Development Lab

Register No.: URK23CS5003

It is hereby certified that this is the Bonafide record of work done by Ms. EVANGELINE ARUNA M during the even semester of the academic year 2025-2026 and submitted for the University Practical Examination held on 21-04-2026.

Faculty-in-charge **Examiner**

Faculty Name: Mr Joms Antony

HEAD OF THE DEPARTMENT

Table of Contents

Ex.No	Date	Name of the Exercise	Page No.	Marks	Signature
1.	20.12.2025	Design a Website using HTML5 [Form and Media Elements].	01		
2.	06.01.2026	Design a Website using HTML5 and Advanced CSS.	05		
3.	13.1.2026	Design a Website using Responsive Web page using Bootstrap	12		
4.	20.01.2026	Design a Website using Javascript DOM elements and Events.	17		
5.	27.01.2026	Design a Website using Javascript Time Functions and Objects.	25		
6.	10.02.2026	Design a Website using Jquery.	30		
7.	17.02.2026	Design a Website using JSON and NoSql.	36		
8.	24.02.2026	Design a Website using NodeJs with a Database module.	38		
9.	17.03.2026	Design a Website using NoSql Database - MongoDB.	45		
10.	24.03.2026	Design a Website using AI Tools.	53		

Ex. No: 1	Design a Website using HTML5 [Form and Media Elements]
20.12.2025	

Aim

To design and develop an Online Event Registration & Promotional Portal using HTML5 that demonstrates the use of form elements and media elements such as images, audio, and video.

Description

The portal allows users to view details of an upcoming event through HTML5 media elements (image banner, promotional video, background audio).Users can then register for the event using an HTML5 form with various input types (text, email, date, radio buttons, select menus, textarea, file upload).

This experiment demonstrates the integration of multimedia content and interactive form elements in a modern HTML5 webpage.

Code

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>Music Concert Registration</title>
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>

<body>
   <h1> Live Music Concert 2026 </h1>
   <h2>Register Now!</h2>
 <!-- Event Banner Image -->
   <img src="banner.png" alt="Music Concert Banner" width="100%">
```

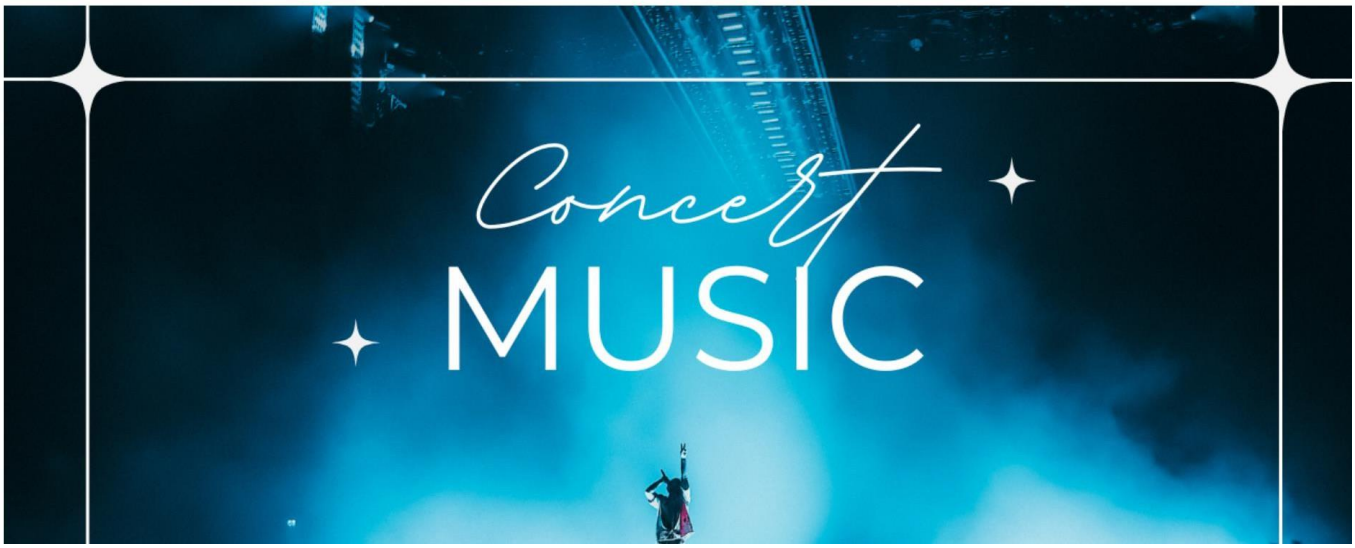
```
<br><br>
 <!-- Promotional Video -->
    <h3>Watch Our Promo Video</h3>
    <video width="600" controls>

<source src="promotional.mp4" type="video/mp4">
        Your browser does not support the video tag.
    </video>
 <br><br>

    <!-- Intro Audio -->
    <h3>Concert Intro Music</h3>
    <audio controls>
        <source src="Intro.mp3" type="audio/mpeg">
        Your browser does not support the audio tag.
    </audio>
    <hr>
    <!-- Registration Form -->
    <h2>Music Concert Registration Form</h2>

    <form action="#" method="post">

        <label>Full Name:</label><br>
        <input type="text" name="fullname" required><br><br>

        <label>Email Address:</label><br>
        <input type="email" name="email" required><br><br>

        <label>Date of Birth:</label><br>
        <input type="date" name="dob" required><br><br>
        <label>Gender:</label><br>
```

```
        <input type="radio" name="gender" value="Male" required> Male
        <input type="radio" name="gender" value="Female"> Female
        <input type="radio" name="gender" value="Other"> Other
        <br><br>

        <label>Ticket Type:</label><br>
        <select name="ticket" required>
            <option value="">--Select Ticket Type--</option>
            <option value="General">General</option>
            <option value="VIP">VIP</option>
            <option value="Backstage">Backstage Pass</option>
        </select>
        <br><br>

<label>Additional Comments:</label><br>
        <textarea name="comments" rows="4" cols="40"></textarea>
        <br><br>

        <label>Upload ID Proof:</label><br>
        <input type="file" name="idproof" accept=".jpg,.png,.pdf" required>
        <br><br>
        <input type="submit" value="Register">
</form>
</body>
</html>
```

Input & Output

🎵 **Live Music Concert 2026** 🎵

Register Now!



Watch Our Promo Video

Concert Intro Music

0:00 / 0:58

Music Concert Registration Form

Full Name:

Email Address:

Date of Birth:
mm / dd / yyyy

Gender:
Male Female Other

Ticket Type:
--Select Ticket Type--

Additional Comments:

Upload ID Proof:
Choose File No file chosen

Register

Result

The experiment was successfully completed.

Ex. No: 2	Design a Website using HTML5 and Advanced CSS
06.01.2026	

Aim

To design and enhance the appearance of an Online Event Registration and Promotional Portal using CSS and Advanced CSS techniques in order to create an attractive, responsive, and user-friendly web interface.

Description

In this experiment, CSS and Advanced CSS concepts are applied to improve the overall look and usability of an Online Event Registration and Promotional Portal. Various styling techniques such as layout design, font styling, color combinations, spacing, alignment, and background customization are implemented to structure the web pages effectively. Advanced features like Flexbox, Grid, transitions, animations, and media queries are used to create responsive and interactive web pages that adapt to different screen sizes. This experiment helps in understanding how proper styling enhances user experience, improves accessibility, and gives a professional appearance to web applications.

Code

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Music Concert Registration - Styled</title>
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <!-- External CSS Linking -->
    <link rel="stylesheet" href="style.css">
</head>
<body>
```

```
<header>
        <h1>Live Music Concert 2026 </h1>
        <p>Experience the Rhythm of Life!</p>
    </header>

    <div class="container">

<!-- Media Section -->
        <div class="media-section">

<img src="banner.png" alt="Music Concert Banner">

            <h3>Promo Video</h3>
            <video controls>
                <source src="promotional.mp4" type="video/mp4">
            </video>

            <h3>Intro Music</h3>
            <audio controls>
                <source src="intro.mp3" type="audio/mpeg">
            </audio>

        </div>

        <!-- Form Section -->
        <div class="form-section">
            <h2>Register Now</h2>
            <form>
                <input type="text" placeholder="Full Name" required>
                <input type="email" placeholder="Email Address" required>
                <input type="date" required>
                <div class="radio-group">
```

```
 <label><input type="radio" name="gender" required> Male</label>
                <label><input type="radio" name="gender"> Female</label>
                <label><input type="radio" name="gender"> Other</label>
            </div>
<select required>
                <option value="">Select Ticket Type</option>
                <option>General</option>
                <option>VIP</option>
                <option>Backstage</option>
            </select>

            <textarea placeholder="Additional Comments"></textarea>

            <input type="file" required>

            <button type="submit">Register</button>
</form>
      </div>
 </div>
</body>
</html>
```

Style.css code

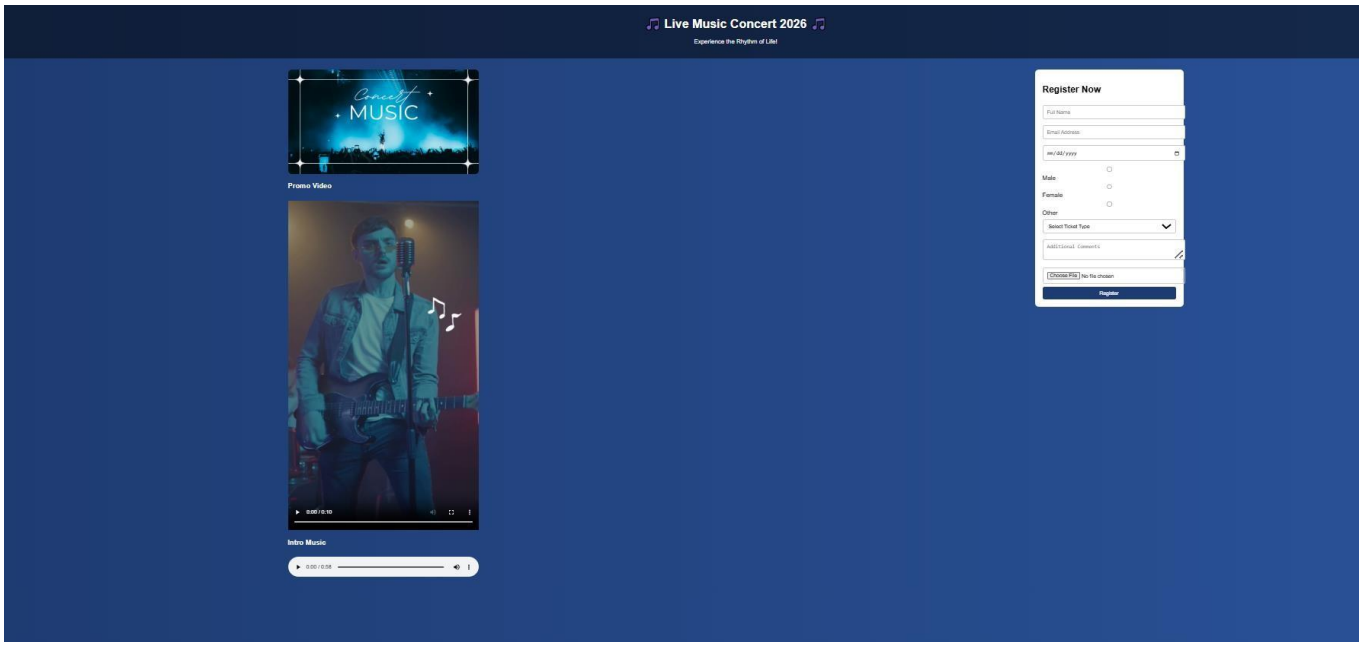
```
/* Page Styling */
body {
    font-family: Arial, sans-serif;
    margin: 0;
    background: linear-gradient(to right, #1e3c72, #2a5298);
    color: white;
}
```

```
/* Header */
header {
    text-align: center;
    padding: 20px;
    background-color: rgba(0,0,0,0.5);
}
/* Main Container using Flexbox */
.container {
    display: flex;
    justify-content: space-around;
    align-items: flex-start;
    padding: 30px;
    gap: 30px;
}
/* Media Section */
.media-section img {
    width: 100%;
    max-width: 500px;
    border-radius: 10px;
}

video, audio {
    width: 100%;
    max-width: 500px;
    margin-top: 10px;
}
/* Form Section */
.form-section {
    background: white;
    color: black;
    padding: 20px;
```

```
border-radius: 10px;
    width: 350px;
}
/* Inputs */
form input,
form select,
form textarea {
    width: 100%;
    padding: 10px;
    margin: 8px 0;
    border-radius: 5px;
    border: 1px solid #ccc;
    transition: 0.3s;
}
/* Input Focus Transition */
form input:focus,
form select:focus,
form textarea:focus {
    border-color: #2a5298;
    box-shadow: 0 0 8px #2a5298;
}
/* Button Styling */
button {
    width: 100%;
    padding: 10px;
    background-color: #1e3c72;
    color: white;
    border: none;
    border-radius: 5px;
    cursor: pointer;
    transition: 0.3s;
```

```
}
/* Button Hover Transition */
button:hover {
    background-color: #2a5298;
    transform: scale(1.05);
}
/* Media Queries - Tablet */
@media (max-width: 768px) {
    .container {
        flex-direction: column;
        align-items: center;
    }
}
/* Media Queries - Mobile */
@media (max-width: 480px) {
    header h1 {
        font-size: 20px;
    }

    .form-section {
        width: 90%;
    }
}
```

Input & Output



Result

The experiment was successfully completed.

Ex. No: 3	Design a Website using Responsive Web page using Bootstrap
13.1.2026	

Aim

To design and develop a responsive website using Bootstrap that adapts seamlessly to different screen sizes and devices by utilizing Bootstrap's grid system, components, and responsive utilities.

Description

The responsive website is developed using the Bootstrap framework to ensure consistency in layout and design across various devices such as desktops, tablets, and mobile phones.Bootstrap's grid system, predefined CSS classes, and UI components are used to create a flexible and responsive layout without writing extensive custom CSS.This experiment demonstrates the effective use of Bootstrap to build modern, mobile-first, and responsive web pages.

Code

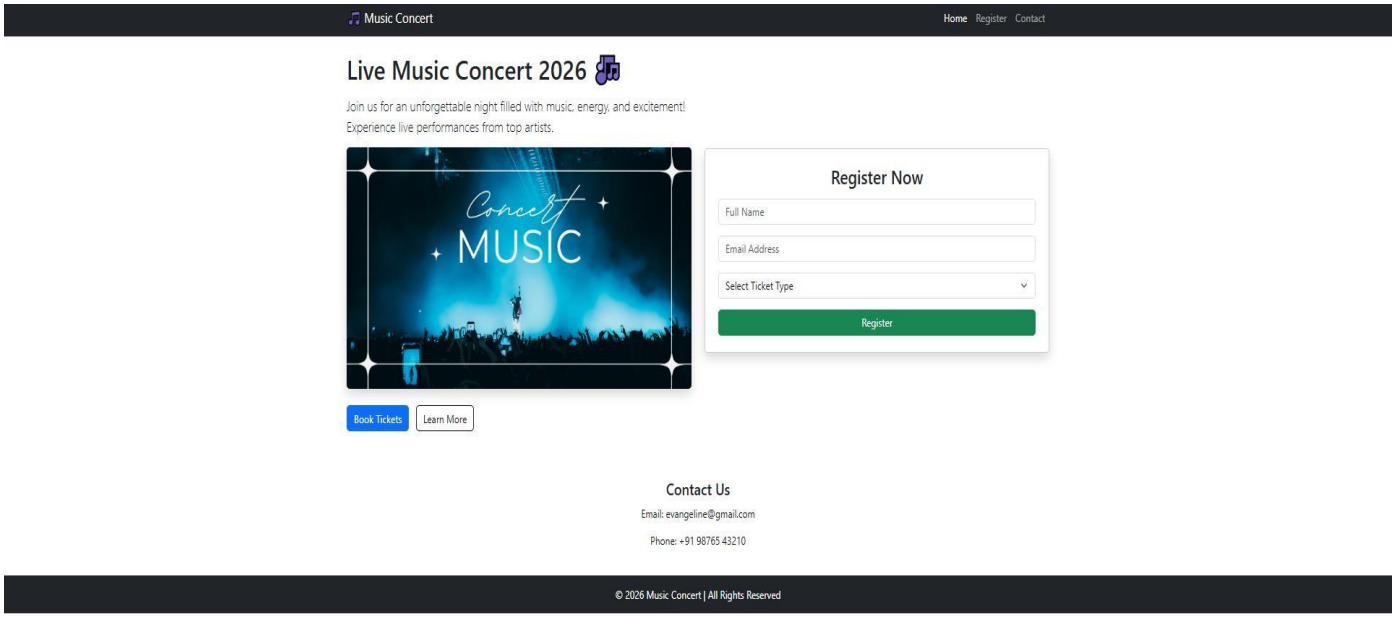
```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>Music Concert - Bootstrap Responsive Website</title>
   <meta name="viewport" content="width=device-width, initial-scale=1">
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
   <style>
      html {
         scroll-behavior: smooth;
      }
```

```
 </style>
</head>

<body>
<nav class="navbar navbar-expand-lg navbar-dark bg-dark fixed-top">
    <div class="container">
        <a class="navbar-brand" href="#">Music Concert</a>
        <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-
target="#navbarNav">
            <span class="navbar-toggler-icon"></span>
        </button>
        <div class="collapse navbar-collapse" id="navbarNav">
            <ul class="navbar-nav ms-auto">
                <li class="nav-item">
                    <a class="nav-link active" href="#">Home</a>
                </li>
                <li class="nav-item">
                    <a class="nav-link" href="#register">Register</a>
                </li>
                <li class="nav-item">
                    <a class="nav-link" href="#contact">Contact</a>
                </li>
            </ul>
        </div>
    </div>
</nav>
<!-- Spacer for fixed navbar -->
<div style="margin-top: 80px;"></div>
<div class="container mt-5">
    <div class="row align-items-center">
```

```
<!-- Left Section -->
    <div class="col-md-6 mb-4">
        <h1 class="mb-3">Live Music Concert 2026 </h1>
        <p class="lead">
            Join us for an unforgettable night filled with music,
            energy, and excitement! Experience live performances
            from top artists.
        </p>
        <!-- Responsive Image -->
        <img src="banner.png" class="img-fluid rounded shadow mb-4" alt="Concert Banner">
        <!-- Buttons -->
        <a href="#register" class="btn btn-primary me-2">Book Tickets</a>
        <a href="#" class="btn btn-outline-dark">Learn More</a>
    </div>
    <!-- Right Section (Register Form) -->
    <div class="col-md-6" id="register">
        <div class="card shadow p-4">
            <h3 class="mb-3 text-center">Register Now</h3>
            <form>
                <div class="mb-3">
                    <input type="text" class="form-control" placeholder="Full Name" required>
                </div>
                <div class="mb-3">
                    <input type="email" class="form-control" placeholder="Email Address" required>
                </div>
                <div class="mb-3">
                    <select class="form-select" required>
                        <option value="">Select Ticket Type</option>
                        <option>General</option>
                        <option>VIP</option>
```

```
<option>Backstage</option>
                    </select>
                </div>
                <button type="submit" class="btn btn-success w-100">
                    Register
                </button>
            </form>
        </div>
    </div>
  </div>
</div>
<div class="container mt-5" id="contact">
  <div class="row">
    <div class="col text-center">
      <h4>Contact Us</h4>
      <p>Email: evangeline@gmail.com</p>
      <p>Phone: +91 98765 43210</p>
    </div>
  </div>
</div>
<footer class="bg-dark text-white text-center p-3 mt-4">
  © 2026 Music Concert | All Rights Reserved
</footer>
<!-- Bootstrap JS CDN -->
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```

Input & Output



Result

The experiment was successfully completed.

Ex. No: 4	Design a Website using Javascript DOM elements and Events
20.01.2026	

Aim

To create a web page which demonstrates dynamic actions for user interactions using JavaScript document object model methods and events.

Description

Document Object Model:

A Document object represents the HTML document that is displayed in that window. The Document object has various properties that refer to other objects which allow access to and modification of document content. The way a document content is accessed and modified is called the Document Object Model, or DOM.

Finding HTML Element by Id

```
var myElement = document.getElementById("intro");
```

Finding HTML Elements by Tag Name

```
var x = document.getElementsByTagName("p");
```

Finding HTML Elements by Class Name

```
var x = document.getElementsByClassName("intro");
```

Finding HTML Elements by CSS Selectors

```
var x = document.querySelectorAll("p.intro");
```

Attach event handler to the document

```
document.addEventListener("click", function()
  {
  document.getElementById("demo").innerHTML = "Karunya";
  });
```

JavaScript Events:

JavaScript's interaction with HTML is handled through events that occur when the user or the browser manipulates a page.

Code

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Music Concert DOM & Events</title>
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <style>
        body {
            font-family: Arial, sans-serif;
            background: #f4f6f8;
            margin: 0;
            padding: 20px;
        }

        h2 {
            text-align: center;
            color: #1f2933;
        }

        .container {
            max-width: 900px;
            margin: auto;
            background: white;
            padding: 20px;
            border-radius: 8px;
        }

        .box {
            border: 1px solid #ccc;
            padding: 15px;
            margin-bottom: 20px;
            border-radius: 6px;
        }
```

```
  label {
        font-weight: bold;
    }

    input, select, button {
        width: 100%;
        padding: 8px;
        margin: 8px 0;
    }

    button {
        background: #2563eb;
        color: white;
        border: none;
        cursor: pointer;
    }

    button:hover {
        background: #1d4ed8;
    }

    .result {
        background: #e5e7eb;
        padding: 10px;
        margin-top: 10px;
        font-weight: bold;
    }

    @media (max-width: 600px) {
        body {
            padding: 10px;
        }
    }
  </style>
</head>
```

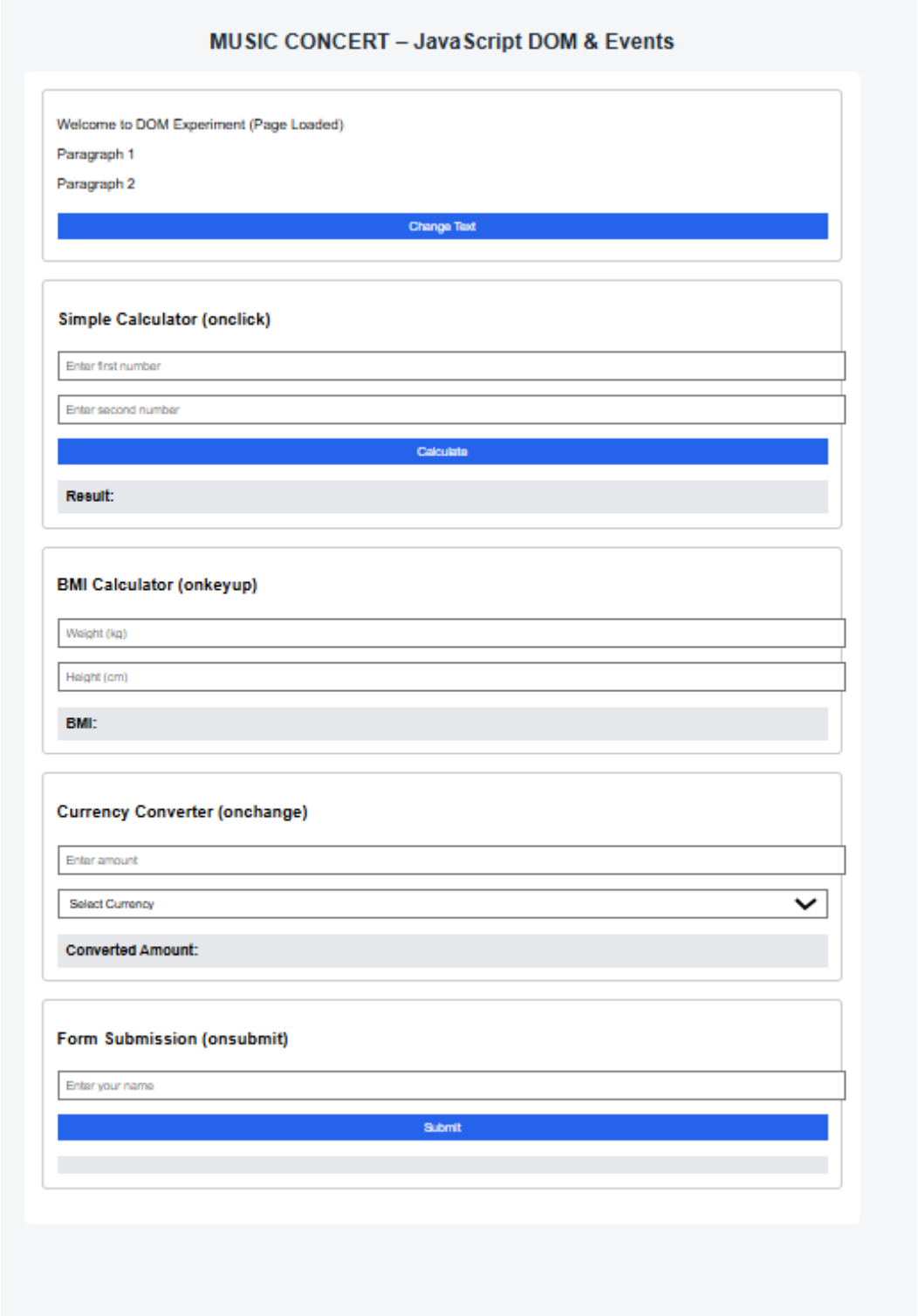
```
<body onload="pageLoaded()">

<h2>MUSIC CONCERT – JavaScript DOM & Events</h2>

<div class="container">

  <!-- TC01, TC02, TC03: DOM Access -->
  <div class="box">
    <p id="intro">Welcome to DOM Experiment</p>
    <p class="demo">Paragraph 1</p>
    <p class="demo">Paragraph 2</p>

    <button onclick="changeText()">Change Text</button>
  </div>

  <!-- TC04, TC08: onclick & Calculation -->
  <div class="box">
    <h3>Simple Calculator (onclick)</h3>

    <input type="number" id="num1" placeholder="Enter first number">
    <input type="number" id="num2" placeholder="Enter second number">

    <button onclick="calculate()">Calculate</button>

    <div class="result" id="calcResult">Result:</div>
  </div>

  <!-- TC04, TC07, TC08: onkeyup & DOM update -->
  <div class="box">
    <h3>BMI Calculator (onkeyup)</h3>

    <input type="number" id="weight" placeholder="Weight (kg)" onkeyup="calculateBMI()">
    <input type="number" id="height" placeholder="Height (cm)" onkeyup="calculateBMI()">

    <div class="result" id="bmiResult">BMI:</div>
  </div>
```

```
  <!-- TC05: onchange -->
    <div class="box">
        <h3>Currency Converter (onchange)</h3>

        <input type="number" id="amount" placeholder="Enter amount">
        <select id="currency" onchange="convertCurrency()">
            <option value="">Select Currency</option>
            <option value="80">USD to INR</option>
            <option value="90">EUR to INR</option>
            <option value="100">GBP to INR</option>
            <option value="1">INR to INR</option>
        </select>

        <div class="result" id="currencyResult">Converted Amount:</div>
    </div>

    <!-- TC05: onsubmit -->
    <div class="box">
        <h3>Form Submission (onsubmit)</h3>

        <form onsubmit="return submitForm()">
            <input type="text" id="username" placeholder="Enter your name">
            <button type="submit">Submit</button>
        </form>

        <div class="result" id="formResult"></div>
    </div>

</div>

<script>
    /* TC06: onload */
    function pageLoaded() {
        document.getElementById("intro").innerHTML += " (Page Loaded)";
    }
```

```
function changeText() {
    document.getElementById("intro").style.color = "blue";

    let items = document.getElementsByClassName("demo");
    for (let i = 0; i < items.length; i++) {
        items[i].innerHTML = "Updated using Class Name";
    }

    let tags = document.getElementsByTagName("p");
    tags[0].style.fontWeight = "bold";
}

/*   TC04,   TC08   */
function calculate() {
    let a = parseFloat(document.getElementById("num1").value);
    let b = parseFloat(document.getElementById("num2").value);
    let result = a + b;

    document.getElementById("calcResult").innerHTML = "Result: " + result;
}

/* TC04, TC07, TC08 */
function calculateBMI() {
    let w = document.getElementById("weight").value;
    let h = document.getElementById("height").value / 100;

    if (w && h) {
        let bmi = (w / (h * h)).toFixed(2);
        let status = "Normal";

        if (bmi < 18) status = "Underweight";
        else if (bmi >= 25 && bmi < 30) status = "Overweight";
        else if (bmi >= 30) status = "Obese";
```

```
 document.getElementById("bmiResult").innerHTML =
            "BMI: " + bmi + " (" + status + ")";
        }
    }

    /* TC05 */
    function convertCurrency() {
        let amount = document.getElementById("amount").value;
        let rate = document.getElementById("currency").value;

        if (amount && rate) {
            document.getElementById("currencyResult").innerHTML =
                "Converted Amount: " + (amount * rate);
        }
    }

    /* TC05 */
    function submitForm() {
        let name = document.getElementById("username").value;
        document.getElementById("formResult").innerHTML =
            "Form submitted by " + name;
        return false; // prevent reload
    }
</script>

</body>
</html>
```

Input & Output



Result

The experiment was successfully completed.

Ex. No: 05	Design a Website using Javascript Time Functions and Objects
27.01.2026	

Aim

To develop a JavaScript Event Countdown Timer that displays the remaining time to a future event and updates dynamically every second.

Description

The application allows users to interact with dynamic web features using JavaScript, including a quiz system and an event countdown timer. Users can attempt multiple-choice questions within a specified time limit, where the interface updates in real time and automatically submits responses when the timer expires. Additionally, the application displays the remaining time for an upcoming event by calculating the difference between the current date and a predefined future date. The countdown is continuously updated every second, and appropriate messages are shown when time limits are reached. This experiment demonstrates the use of JavaScript timing functions such as setInterval() and clearInterval(), along with DOM manipulation to create interactive and time-based web applications.

Code

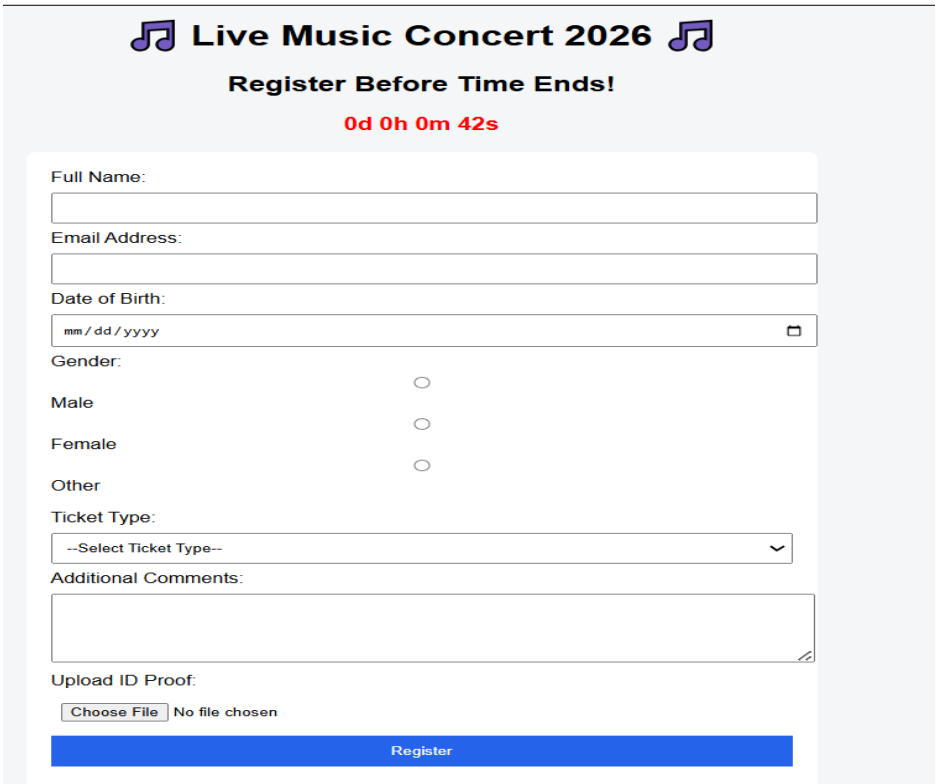
```
<!DOCTYPE html>
<html>
<head>
   <title>Quiz + Event Countdown</title>
</head>
<body>

<h2>Event Countdown Timer</h2>
<p id="eventTimer"></p>

<hr>

<h2>Quiz App</h2>
```

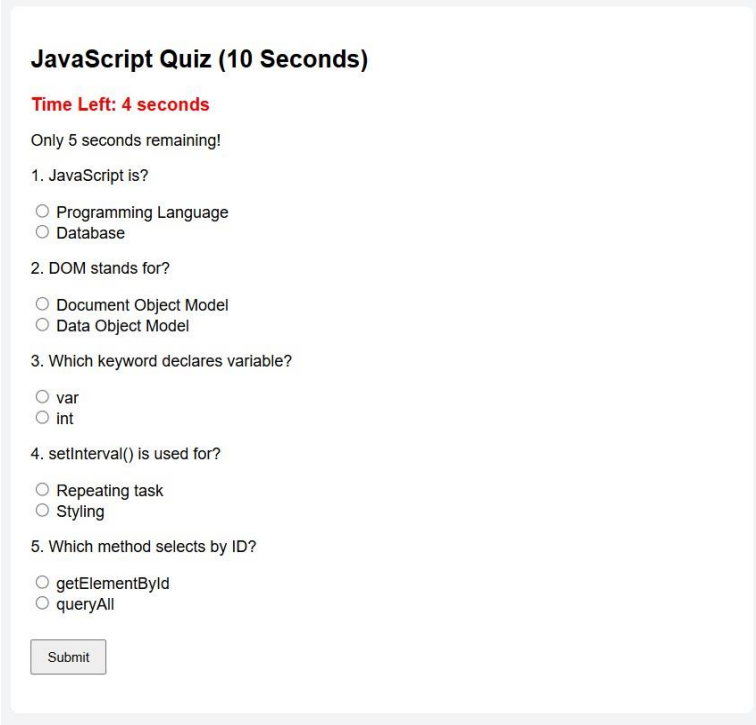
```
<p id="quizTimer">Time: 10</p>
<p id="warning" style="color:red;"></p>

<form id="quizForm">

<p>1. Capital of India?</p>
<input type="radio" name="q1" value="a"> Delhi
<input type="radio" name="q1" value="b"> Mumbai

<p>2. 2 + 2 = ?</p>
<input type="radio" name="q2" value="a"> 4
<input type="radio" name="q2" value="b"> 5

<p>3. Sun rises from?</p>
<input type="radio" name="q3" value="a"> East
<input type="radio" name="q3" value="b"> West

<p>4. HTML stands for?</p>
<input type="radio" name="q4" value="a"> Hyper Text Markup Language
<input type="radio" name="q4" value="b"> High Text Machine Language

<p>5. JS is used for?</p>
<input type="radio" name="q5" value="a"> Styling
<input type="radio" name="q5" value="b"> Logic

<br><br>
<button type="button" onclick="submitQuiz()">Submit</button>

</form>

<p id="result"></p>

<script>
/* ================= EVENT COUNTDOWN ================= */

let eventDate = new Date("April 10, 2026 12:00:00").getTime();

let eventTimer = setInterval(function() {

    let now = new Date().getTime();
    let distance = eventDate - now;
```

```
    let days = Math.floor(distance / (1000 * 60 * 60 * 24));
    let hours = Math.floor((distance % (1000 * 60 * 60 * 24)) / (1000 * 60 * 60));
    let minutes = Math.floor((distance % (1000 * 60 * 60)) / (1000 * 60));
    let seconds = Math.floor((distance % (1000 * 60)) / 1000);

    document.getElementById("eventTimer").innerHTML =
        days + "d " + hours + "h " + minutes + "m " + seconds + "s ";

    if (distance < 0) {
        clearInterval(eventTimer);
        document.getElementById("eventTimer").innerHTML = "Registration Closed";
    }

}, 1000);


/* ================= QUIZ TIMER ================= */

let time = 10;
let quizTimer = setInterval(updateQuizTimer, 1000);

function updateQuizTimer() {
    document.getElementById("quizTimer").innerText = "Time: " + time;

    if (time == 5) {
        document.getElementById("warning").innerText = "Only 5 seconds remaining!";
    }

    if (time == 0) {
        clearInterval(quizTimer);
        submitQuiz();
    }
    time--;
}
```

```
/* ================= QUIZ LOGIC ================= */

function submitQuiz() {
    clearInterval(quizTimer);

    let answers = {
        q1: "a",
        q2: "a",
        q3: "a",
        q4: "a",
        q5: "b"
    };

    let score = 0;
    let form = document.forms["quizForm"];

    for (let q in answers) {
        if (form[q].value == answers[q]) {
            score++;
        }
    }

    document.getElementById("result").innerText = "Score: " + score;

    let inputs = document.querySelectorAll("input");
    inputs.forEach(i => i.disabled = true);
}
</script>

</body>
</html>
```

Input & Output





Result

The experiment was successfully completed.

Ex. No: 06	Design a Website using Jquery
10.02.2026	

1. Interactive Event Details Display using jQuery

Aim

To develop an interactive event display webpage using jQuery that dynamically shows event details, applies effects, and passes selected event data to a registration form.

Description

The application allows users to view multiple events displayed on a webpage, where each event contains a title, date, and hidden additional details. When the user clicks the “Show Details” button, the hidden content is displayed using jQuery effects, and the button text toggles between “Show Details” and “Hide Details”. The selected event is highlighted by changing its background color, and the corresponding event title is accessed using traversing methods and displayed in an alert box. The user can click the “Register” button to navigate to a registration form page, where the selected event name is automatically passed and displayed in a non-editable text field. This experiment demonstrates the use of jQuery selectors, DOM manipulation, event handling, effects like slideToggle(), traversing methods, CSS manipulation, and data transfer between webpages.

Code

```
<!DOCTYPE html>
<html>
<head>
   <title>Event Details</title>
   <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
   <style>
      .event { border:1px solid black; padding:10px; margin:10px; }
      .details { display:none; }
   </style>
</head>
```

```
<body>

<div class="event">
    <h3>Tech Fest</h3>
    <p>Date: April 10</p>
    <div class="details">
        <p>Technical events and coding contests</p>
        <button class="register">Register</button>
    </div>
    <button class="toggle">Show Details</button>
</div>

<div class="event">
    <h3>Music Night</h3>
    <p>Date: April 12</p>
    <div class="details">
        <p>Live music performances</p>
        <button class="register">Register</button>
    </div>
    <button class="toggle">Show Details</button>
</div>

<script>
$(".toggle").click(function() {
    let eventDiv = $(this).parent();

    eventDiv.find(".details").slideToggle();

    let btn = $(this);
    btn.text(btn.text() === "Show Details" ? "Hide Details" : "Show Details");

    $(".event").css("background", "white");
    eventDiv.css("background", "lightblue");

    let title = eventDiv.find("h3").text();
    eventDiv.find("h3").css("color", "red");

    alert("Event: " + title);
});
```

```
$(".register").click(function() {
    let title = $(this).closest(".event").find("h3").text();
    window.location.href = "form.html?event=" + encodeURIComponent(title);
});
</script>

</body>
</html>
```

2. Interactive Event Registration Form using jQuery

Aim

To design and validate an event registration form using **jQuery**, ensuring proper input validation and dynamic user feedback.

Description

The application provides an event registration form where users can enter their details such as name, email, selected event name, and mobile number. The event name is automatically populated from the previous page and displayed in a non-editable field. When the user submits the form, jQuery is used to retrieve and validate all inputs. If any field is empty or invalid, appropriate error messages are displayed in red and the corresponding fields are highlighted. Email format and mobile number constraints are validated using conditions. If all inputs are valid, a success message is displayed in green, and the submit button is disabled and updated to “Registered”. The reset button clears all inputs and messages. This experiment demonstrates jQuery selectors, form validation, DOM manipulation, CSS styling, event handling, and attribute manipulation.

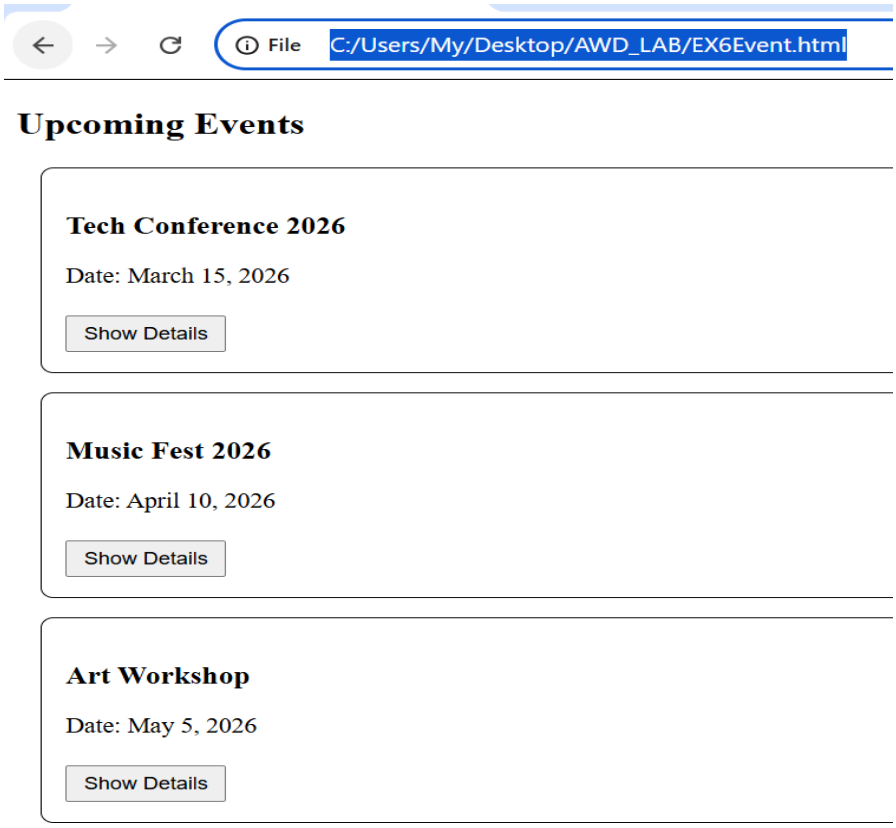
Code

```
<!DOCTYPE html>
<html>
<head>
    <title>Registration Form</title>
    <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
</head>
<body>

<h2>Event Registration</h2>
```

```
<form id="regForm">
    Name: <input type="text" id="name"><br><br>
    Email: <input type="text" id="email"><br><br>
    Event: <input type="text" id="event" readonly><br><br>
    Mobile: <input type="text" id="mobile"><br><br>

    <button type="button" id="submit">Submit</button>
    <button type="button" id="reset">Reset</button>
</form>

<p id="msg"></p>

<script>
// Get event name from URL
let params = new URLSearchParams(window.location.search);
$("#event").val(params.get("event"));

$("#submit").click(function()  {
    let name = $("#name").val();
    let email = $("#email").val();
    let mobile = $("#mobile").val();

    let valid = true;
    $("#msg").text("");

    if(name === "" || email === "" || mobile === "") {
        $("#msg").text("All fields required").css("color","red");
        valid = false;
    }

    let emailPattern = /^[^ ]+@[^ ]+\.[a-z]{2,3}$/;
    if(!email.match(emailPattern)) {
        $("#msg").text("Invalid Email").css("color","red");
        $("#email").css("border","2px solid red");
        valid = false;
    }

    if(!/^\d{10,}$/.test(mobile)) {
        $("#msg").text("Invalid Mobile").css("color","red");
        $("#mobile").css("border","2px solid red");
        valid = false;
    }
```

```
 if(valid) {
        $("#msg").text("Registration Successful for " + name)
                .css("color","green");

        $("#submit").prop("disabled", true).text("Registered");
    }
});

$("#reset").click(function() {
    $("#regForm")[0].reset();
    $("#msg").text("");
    $("input").css("border","");
});
</script>

</body>
</html>
```

Input & Output



Upcoming Events

Tech Conference 2026

Date: March 15, 2026

Hide Details

Location: City Hall

Speaker: Industry Experts

Register

Event Registration Form

Participant Name:

Evangeline

Email:

evangelinearuna@karunya.

Event Name:

Music Fest 2026

Mobile Number:

8778435264

Registered Reset

Registration Successful for Evangeline

Result

The experiment was successfully completed.

Ex. No: 07	Design a Website using JSON and NoSql
17.02.2026	

Aim

To develop a simple web server using Node.js that displays a greeting message along with the current date and time in a web browser.

Description

The application allows users to access a simple web server created using Node.js, where a greeting message along with the current system date and time is displayed in the browser. When the user enters the URL in the browser, the server processes the request and dynamically generates the response using the built-in HTTP module. The date and time are retrieved from the system and updated each time the page is refreshed. This experiment demonstrates the basic working of a Node.js server, handling HTTP requests and responses, and generating dynamic content.

Code

```
const http = require("http");

const server = http.createServer((req, res) => {
   const name = "Evangeline"; // put your name
   const currentTime = new Date();

   res.writeHead(200, { "Content-Type": "text/html" });

   res.write(`<h2>Hello from ${name}'s Node.js Server</h2>`);
   res.write(`<p>Current Date and Time: ${currentTime}</p>`);

   res.end();
});

server.listen(3000, "0.0.0.0", () => {
   console.log("Server running at http://localhost:3000");
});
```

Input & Output

Result

The experiment was successfully completed.

Ex. No: 08	Design a Website using NodeJs with a Database module
24.02.2026	

Aim

To develop a web-based event registration system using Node.js, Express, and MySQL that allows users to register for events and enables administrators to search registration details.

Description

The application allows users to register for multiple events through an HTML form by entering their register number, name, and selecting events from given options. The form data is sent to a Node.js server built using Express, where validation is performed to ensure that no more than three events are selected and duplicate registrations are avoided. The validated data is then stored in a MySQL database. The system also provides an admin search feature where the administrator can retrieve registration details using a register number. If the entered register number exists, the corresponding details are displayed; otherwise, an appropriate message is shown. This experiment demonstrates full-stack integration involving frontend form handling, backend processing, database storage, validation, and data retrieval using Node.js and MySQL.

Code

index.html

```
<!DOCTYPE html>
<html>
<head>
   <title>Event Registration</title>
</head>
<body>

<h2>Event Registration Form</h2>
```

```
<form action="/register" method="POST">
Register Number:
    <input type="text" name="regno" required><br><br>

    Name:
    <input type="text" name="name" required><br><br>

    Select Events (Max 3):<br>

    <input type="checkbox" name="events" value="Coding Challenge"> Coding Challenge<br>
    <input type="checkbox" name="events" value="Debugging Contest"> Debugging Contest<br>
    <input type="checkbox" name="events" value="Web Design"> Web Design<br>
    <input type="checkbox" name="events" value="Quiz Competition"> Quiz Competition<br>
    <input type="checkbox" name="events" value="Paper Presentation"> Paper Presentation<br><br>

    <button type="submit">Register</button>

</form>

</body>
</html>
```

Search.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Search Registration</title>
</head>
<body>

<h2>Search Registration</h2>

<form action="/search" method="POST">
    Enter Register Number:
    <input type="text" name="regno" required>
    <button type="submit">Search</button>
</form>

</body>
</html>
```

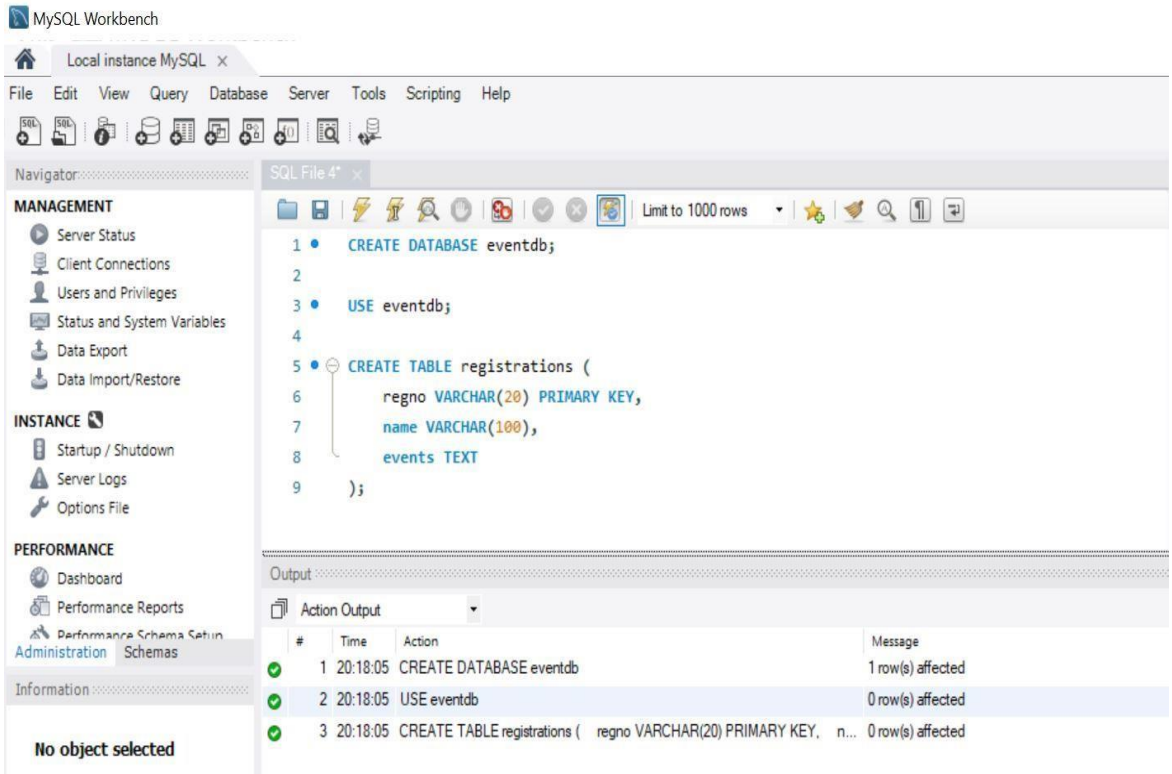
Server.js

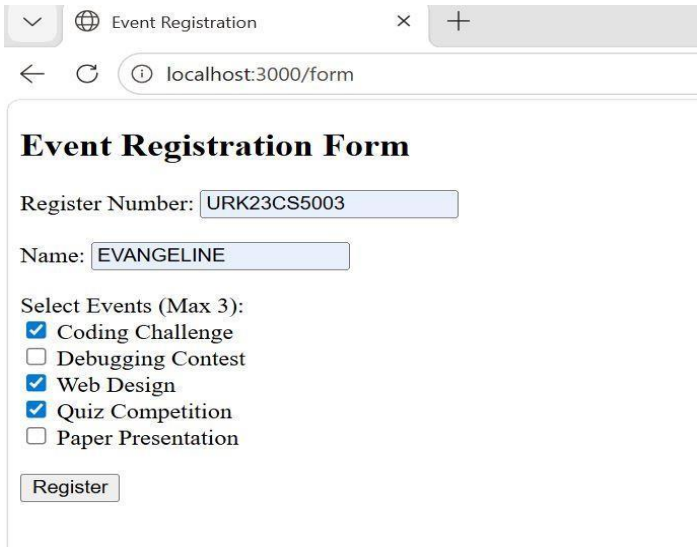
```
const express = require('express');
const bodyParser = require('body-parser');
const mysql = require('mysql2');
const path = require('path');

const app = express();
const port = 3000;

app.use(bodyParser.urlencoded({ extended: true }));

// MySQL connection
const db = mysql.createConnection({
    host: 'localhost',
    user: 'root',
    password: 'aruna',
    database: 'eventdb'
});

db.connect((err) => {
    if (err) {
        console.log("Error connecting to MySQL:", err);
    } else {
        console.log("Connected to MySQL");
    }
});

// Home route
app.get('/', (req, res) => {
    res.send("Server + MySQL Connected!");
});

app.get('/form', (req, res) => {
    res.sendFile(__dirname + '/index.html');
});

app.post('/register', (req, res) => {
    const { regno, name, events } = req.body;
```

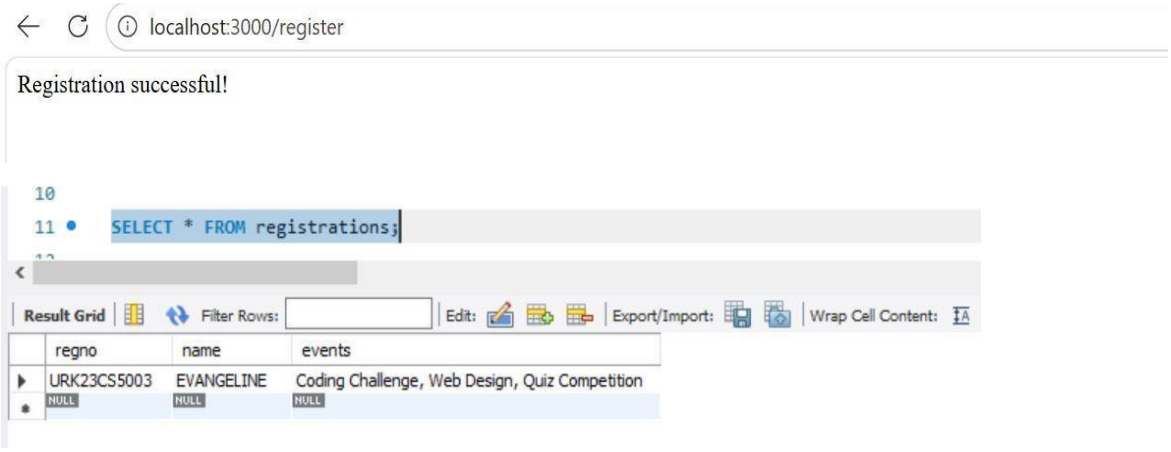
```
if (!events) {
        return res.send("Error: Please select at least one event");
    }

    let eventList = events;

    if (!Array.isArray(eventList)) {
        eventList = [eventList];
    }

    if (eventList.length > 3) {
        return res.send("Error: Maximum 3 events allowed");
    }

    const eventString = eventList.join(", ");

    const sql = "INSERT INTO registrations (regno, name, events) VALUES (?, ?, ?)";

    db.query(sql, [regno, name, eventString], (err, result) => {
        if (err) {
            if (err.code === 'ER_DUP_ENTRY') {
                return res.send("Error: Register number already exists");
            }
            return res.send("Database error");
        }

        res.send("Registration successful!");
    });
});

app.get('/search', (req, res) => {
    res.sendFile(__dirname + '/search.html');
});

app.post('/search', (req, res) => {
    const regno = req.body.regno;

    const sql = "SELECT * FROM registrations WHERE regno = ?";
```

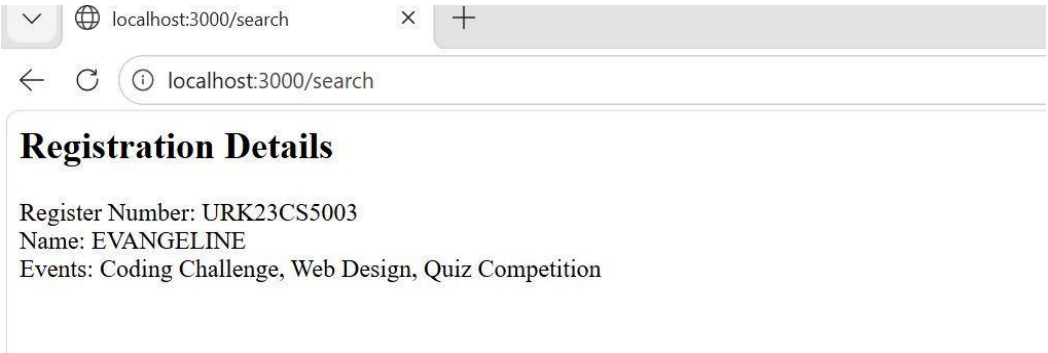
```
db.query(sql, [regno], (err, results) => {
        if (err) {
            return res.send("Database error");
        }

        if (results.length === 0) {
            return res.send("No record found");
        }

        const data = results[0];

        res.send(`
            <h2>Registration Details</h2>
            Register Number: ${data.regno} <br>
            Name: ${data.name} <br>
            Events: ${data.events}
        `);
    });
});

// Start server
app.listen(port, () => {
    console.log(`Server running at http://localhost:${port}`);
});
```

Input & Output









Result

The experiment was successfully completed.

Ex. No: 09	Design a Website using NoSql Database - MongoDB
17.03.2026	

Aim

To develop a web-based event registration system using Node.js, Express, and MongoDB that allows users to register for events and enables administrators to search registration details using a NoSQL database.

Description

The application allows users to register for events by entering their register number, name, and selecting up to three events through an HTML form. The data is sent to a Node.js server built using Express, where validation is performed to ensure that no more than three events are selected and duplicate registrations are prevented. The validated data is stored in a MongoDB database as documents, where each record contains the register number, name, and selected events. The system also provides an admin search feature that allows retrieval of registration details using a register number. If the record exists, the details are displayed; otherwise, an appropriate message is shown. This experiment demonstrates the use of a NoSQL database, document-based data storage, backend processing, validation, and data retrieval using Node.js and MongoDB.

Code

Index.html

```
<!DOCTYPE html>
<html>
<head>
   <title>Event Registration</title>
</head>
<body>
<h2>Event Registration Form</h2>


<form action="/register" method="POST">
```

```
 Register Number:
    <input type="text" name="regno" required><br><br>

    Name:
    <input type="text" name="name" required><br><br>

    Select Events (Max 3):<br>

    <input type="checkbox" name="events" value="Coding Challenge"> Coding Challenge<br>
    <input type="checkbox" name="events" value="Debugging Contest"> Debugging Contest<br>
    <input type="checkbox" name="events" value="Web Design"> Web Design<br>
    <input type="checkbox" name="events" value="Quiz Competition"> Quiz Competition<br>
    <input type="checkbox" name="events" value="Paper Presentation"> Paper Presentation<br><br>

    <button type="submit">Register</button>

</form>

</body>
</html>
```

Search.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Admin Search</title>
</head>
<body>

<h2>Search Registration</h2>

<form action="/search" method="POST">
    Enter Register Number:
    <input type="text" name="regno" required>
    <button type="submit">Search</button>
</form>

</body>
</html>
```

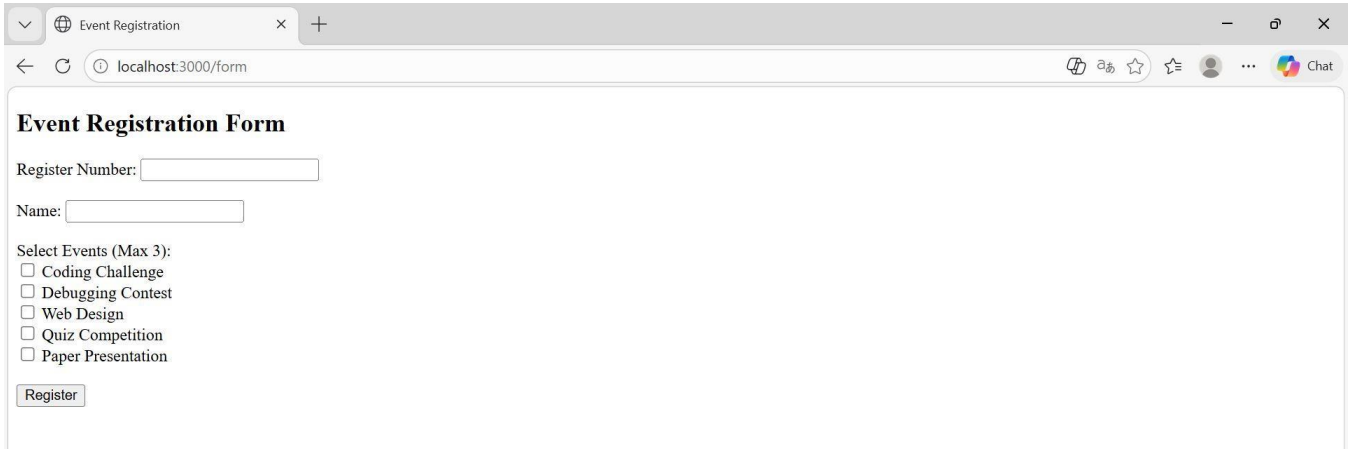
Server.js

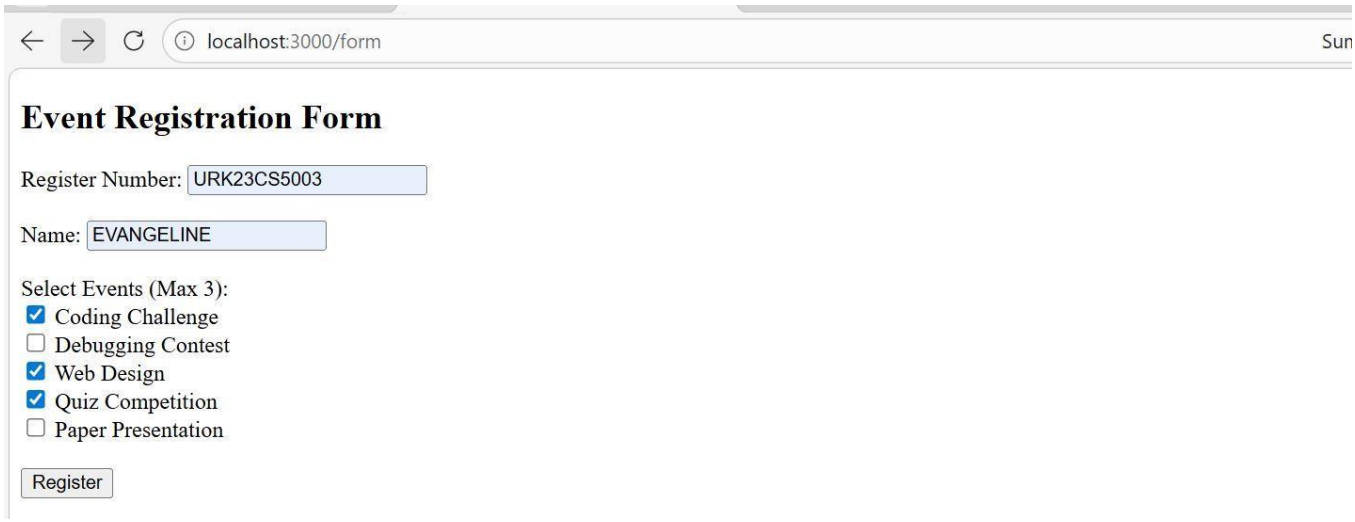
```
const express = require('express');
const bodyParser = require('body-parser');
const { MongoClient } = require('mongodb');
const path = require('path');

const app = express();
const port = 3000;

app.use(bodyParser.urlencoded({ extended: true }));

// MongoDB URL
const url = "mongodb://localhost:27017";
const client = new MongoClient(url);

const dbName = "eventdb";

// Connect to MongoDB
async function connectDB() {
    try {
        await client.connect();
        console.log("Connected to MongoDB");
    } catch (err) {
        console.error("Error connecting to MongoDB:", err);
    }
}
connectDB();


// ---------- ROUTES ---------- //

// Home route
app.get('/', (req, res) => {
    res.send("Server + MongoDB Connected!");
});
```

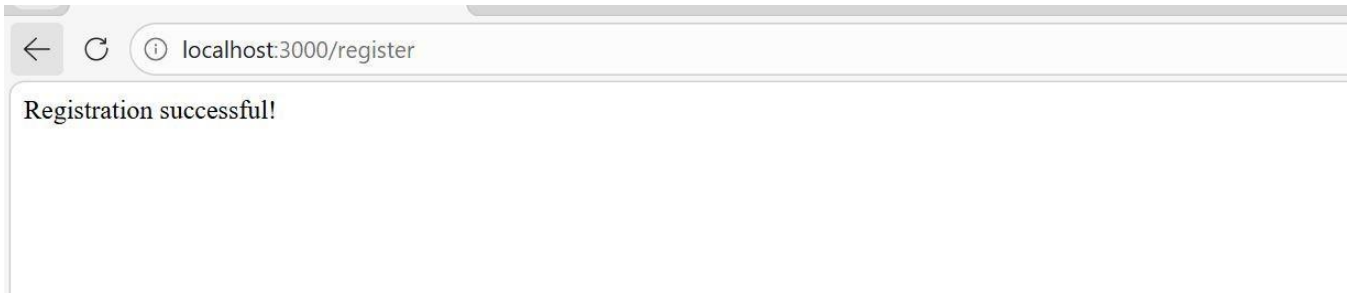
```
// Show registration form
app.get('/form', (req, res) => {
    res.sendFile(__dirname + '/index.html');
});

// Handle registration
app.post('/register', async (req, res) => {
    const { regno, name, events } = req.body;

    const db = client.db(dbName);
    const collection = db.collection("registrations");

    let eventList = events;

    // Convert to array
    if (!Array.isArray(eventList)) {
        eventList = [eventList];
    }

    // Max 3 validation
    if (eventList.length > 3) {
        return res.send("Error: You can select maximum 3 events only.");
    }

    // Duplicate check
    const existing = await collection.findOne({ regno: regno });
    if (existing) {
        return res.send("Error: Register number already exists.");
    }

    // Insert data
    await collection.insertOne({
        regno: regno,
        name: name,
        events: eventList
    });

    res.send("Registration successful!");
});
```

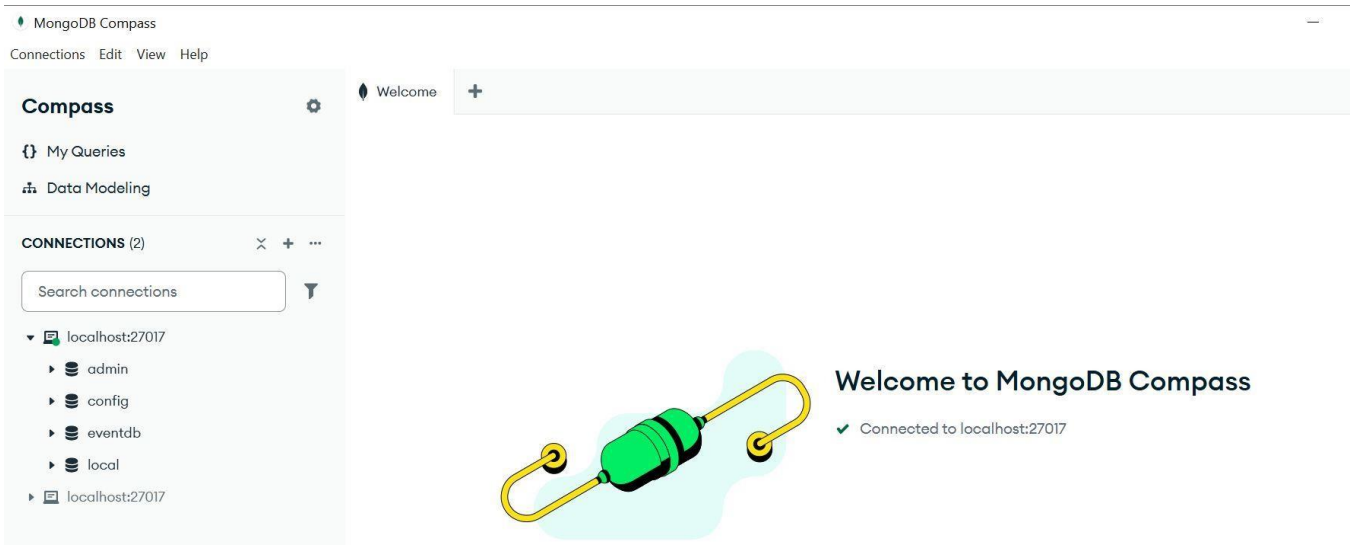
```
// Show admin search page
app.get('/search', (req, res) => {
    res.sendFile(__dirname + '/search.html');
});


// Handle search
app.post('/search', async (req, res) => {
    const regno = req.body.regno;

    const db = client.db(dbName);
    const collection = db.collection("registrations");

    const result = await collection.findOne({ regno: regno });

    if (!result) {
        return res.send("No record found");
    }

    res.send(`
        <h2>Registration Details</h2>
        Register Number: ${result.regno} <br>
        Name: ${result.name} <br>
        Events: ${result.events.join(", ")}
    `);
});


// ---------- START SERVER ---------- //

app.listen(port, () => {
    console.log(`Server running at http://localhost:${port}`);
});
```

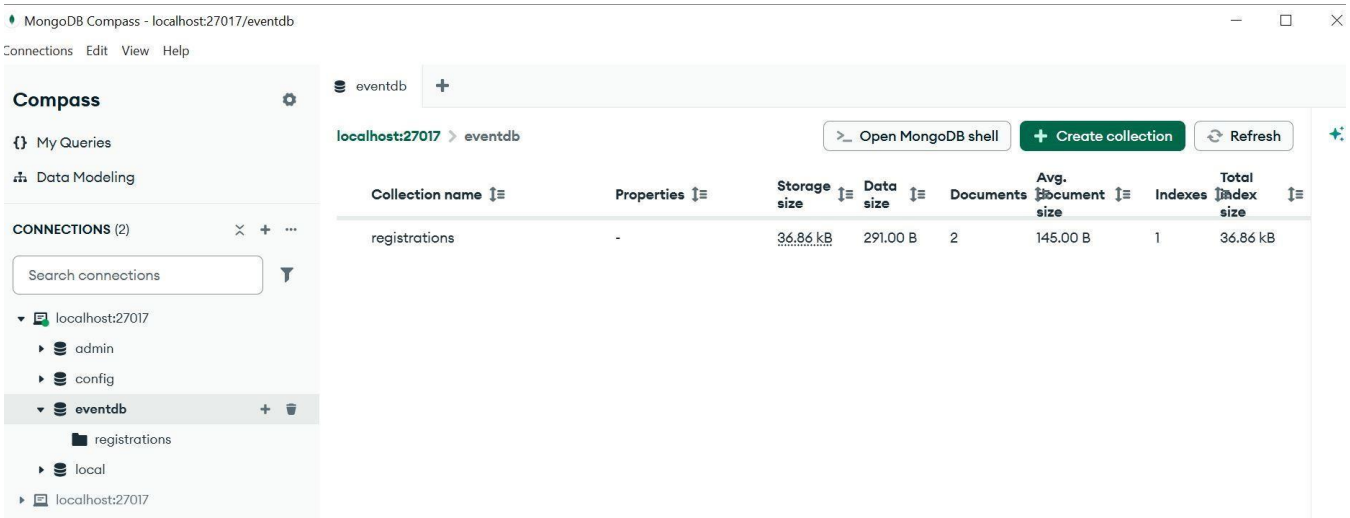
Input & Output

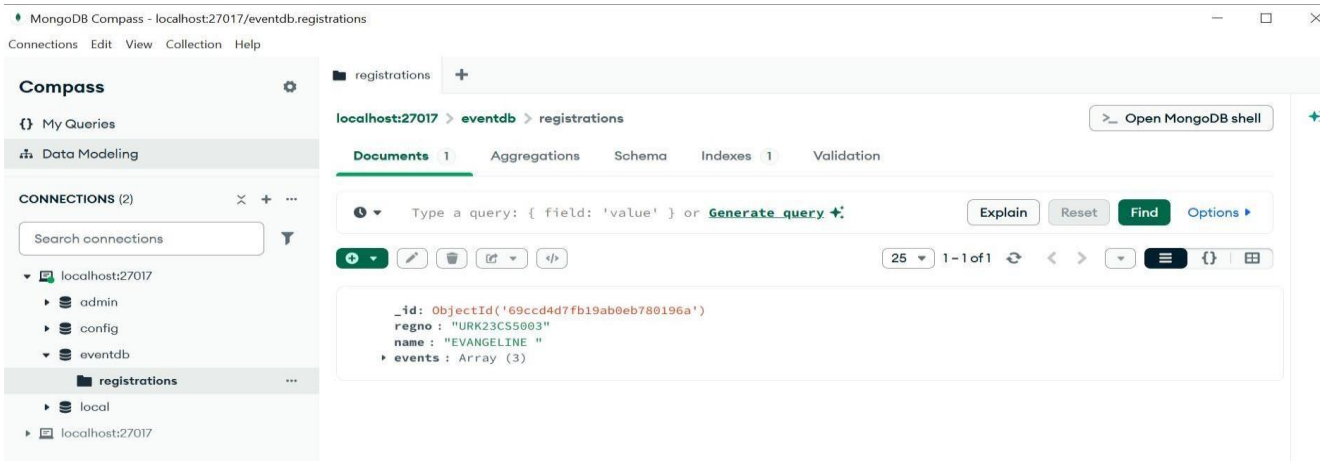


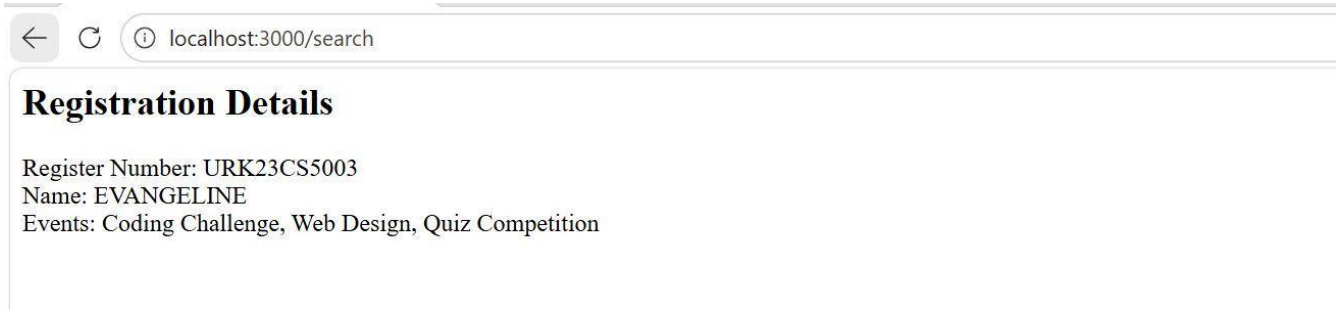





MongoDB Compass
Connections Edit View Help
Compass
My Queries
Data Modeling
CONNECTIONS (2)
Search connections
localhost:27017
admin
config
eventdb
local
localhost:27017
Welcome
Welcome to MongoDB Compass
Connected to localhost:27017


MongoDB Compass - localhost:27017/eventdb
Connections Edit View Help
Compass
My Queries
Data Modeling
CONNECTIONS (2)
Search connections
localhost:27017
admin
config
eventdb
registrations
local
localhost:27017
eventdb
localhost:27017 > eventdb
Open MongoDB shell
Create collection
Refresh
Collection name
Properties
Storage size
Data size
Documents
Avg. Document size
Indexes
Total Index size
registrations
-
36.86 kB
291.00 B
2
145.00 B
1
36.86 kB


MongoDB Compass - localhost:27017/eventdb.registrations
Connections Edit View Collection Help
Compass
My Queries
Data Modeling
CONNECTIONS (2)
Search connections
localhost:27017
admin
config
eventdb
registrations
local
localhost:27017
registrations
localhost:27017 > eventdb > registrations
Open MongoDB shell
Documents 1
Aggregations
Schema
Indexes 1
Validation
Type a query: { field: 'value' } or Generate query
Explain
Reset
Find
Options
25
1-1 of 1
_id: ObjectId('69ccd4d7fb19ab0eb780196a')
regno : "URK23CS5003"
name : "EVANGELINE "
events : Array (3)



Result

The experiment was successfully completed.

Ex. No: 10	Design a Website using AI Tools
24.03.2026	

Aim

To design and develop a responsive website using AI-powered tools for generating layout, content, and code efficiently.

Description

The application involves designing a responsive website using various AI-powered tools to assist in layout creation, content generation, and code development. AI tools such as ChatGPT, GitHub Copilot, and design platforms like Canva or Figma are used to generate HTML structure, CSS styling, and JavaScript functionality. The website includes multiple pages such as Home, About, Services, and Contact, and incorporates responsive design techniques to ensure compatibility across different devices. Dynamic features are added using JavaScript to enhance interactivity. The use of AI tools helps in improving productivity, reducing development time, and enhancing design quality. This experiment demonstrates the integration of artificial intelligence in modern web development processes.

Code

Index.html

```
<!DOCTYPE html>
<html>
<head>
   <title>My Portfolio</title>
   <link rel="stylesheet" href="style.css">
</head>
<body>

<header>
   <h1>Welcome to My Portfolio</h1>
   <p>I am Evangeline - Web Developer</p>
</header>
```

```html
<nav>
    <a onclick="showSection('home')">Home</a>
    <a onclick="showSection('about')">About</a>
    <a onclick="showSection('services')">Skills</a>
    <a onclick="showSection('contact')">Contact</a>
</nav>

<section id="home" class="section">
    <h2>Home</h2>
    <p>This website is created using AI tools like ChatGPT.</p>
    <button onclick="showMsg()">Click Me</button>
</section>

<section id="about" class="section">
    <h2>About Me</h2>
    <p>I am a passionate student with strong interest in Web Development and Artificial Intelligence.</p>
    <p>I enjoy building creative websites and exploring new technologies.</p>
    <p>My goal is to become a skilled full-stack developer.</p>
</section>

<section id="services" class="section">
    <h2>My Skills</h2>
    <ul>
        <li>HTML, CSS, JavaScript</li>
        <li>Java Programming</li>
        <li>Database (MySQL)</li>
    </ul>
</section>

<section id="contact" class="section">
    <h2>Contact</h2>
    <p>Email: evangeline@karunya.edu.in</p>
</section>

<script src="script.js"></script>

</body>
</html>
```

Script.js

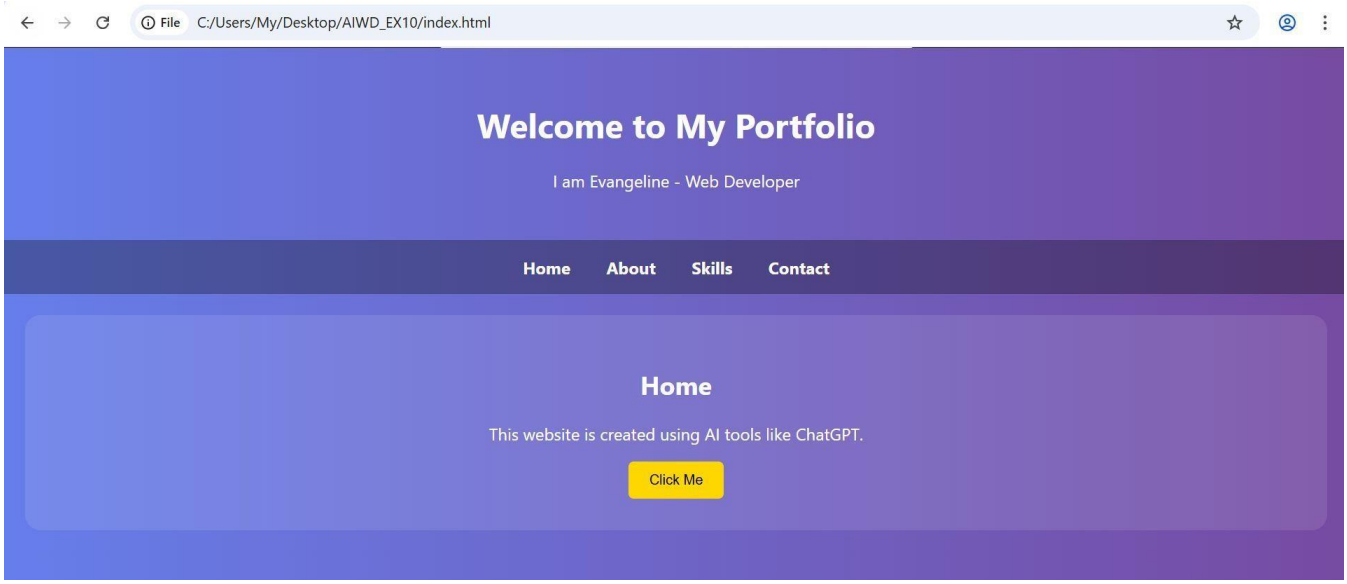
```
function showMsg() {
    alert("Welcome! This portfolio is created using AI tools.");
}

function showSection(sectionId) {
    let sections = document.querySelectorAll(".section");

    sections.forEach(section => {
        section.style.display = "none";
    });

    document.getElementById(sectionId).style.display = "block";
}

// Show home section by default
window.onload = function() {
    showSection('home');
}
```

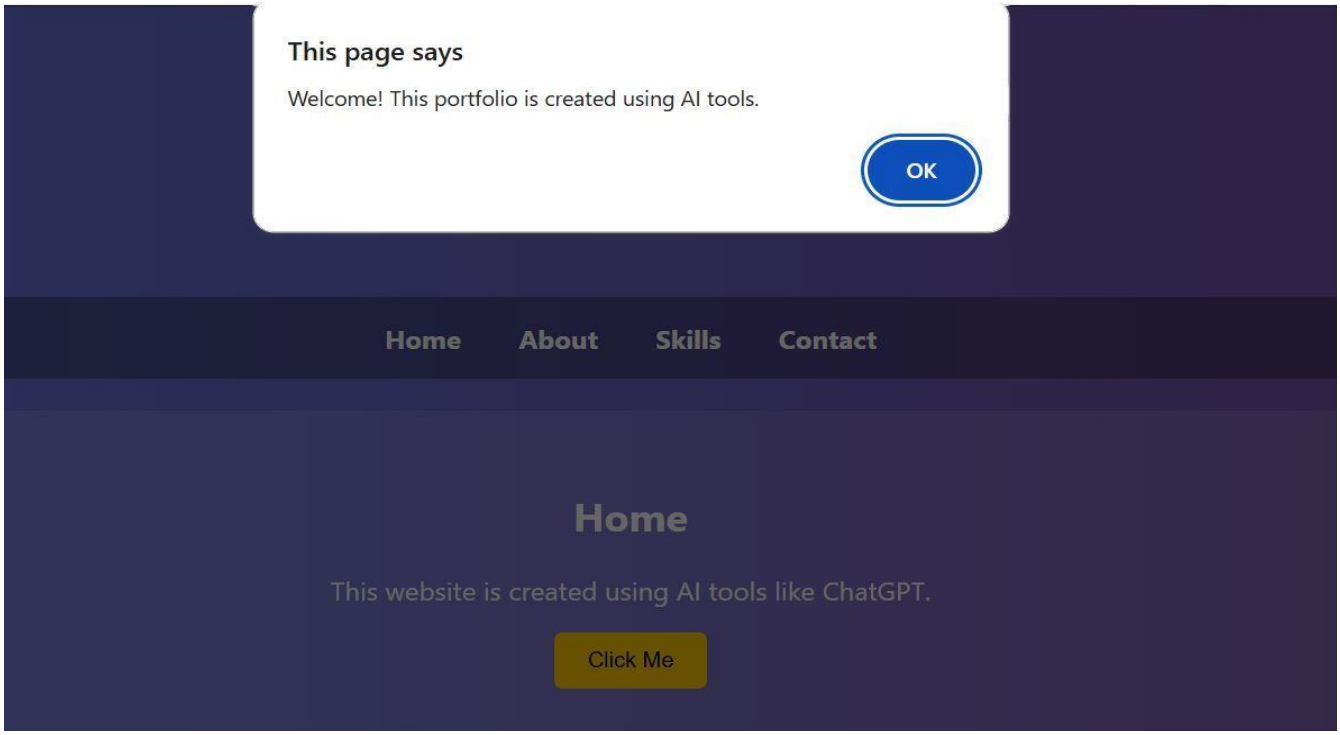
Style.css

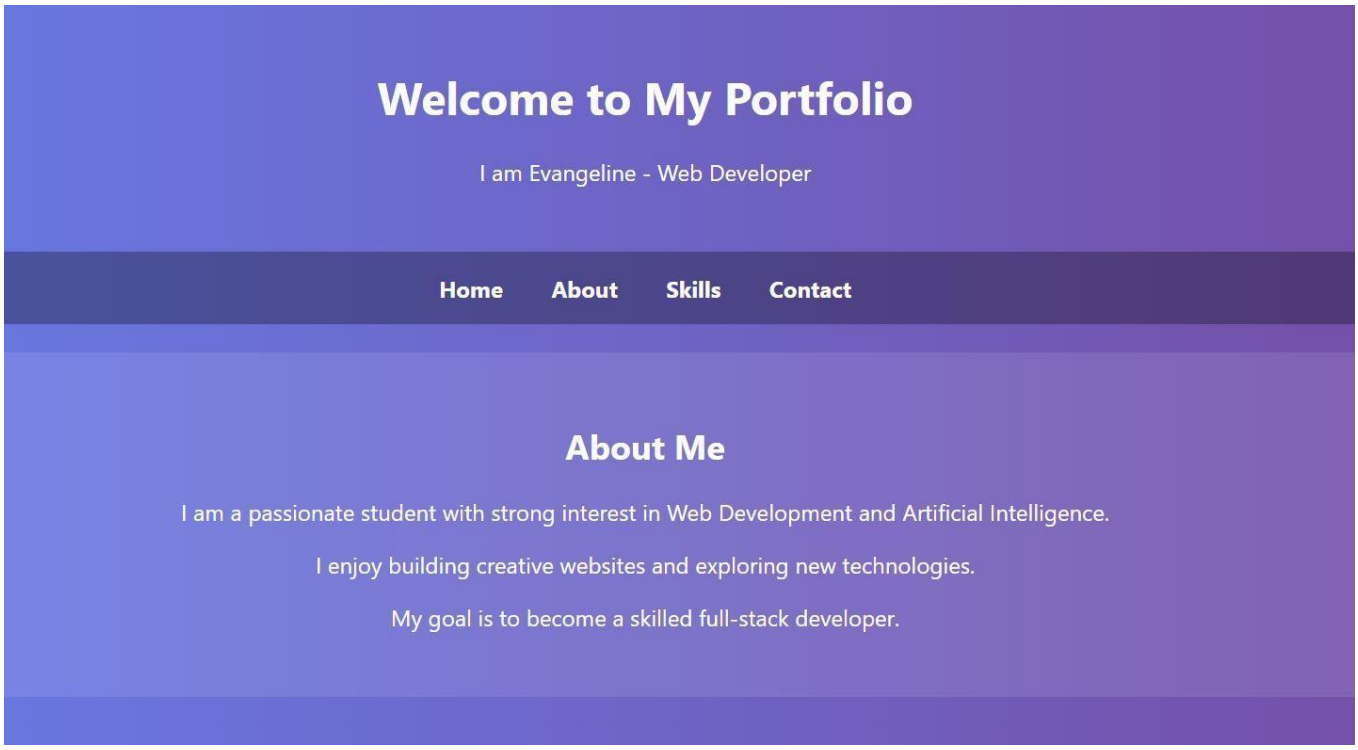
```
body {
    font-family: 'Segoe UI', sans-serif;
    margin: 0;
    background: linear-gradient(to right, #667eea, #764ba2);
    color: white;
    text-align: center;
}

header {
    padding: 30px;
}

nav {
    background: rgba(0,0,0,0.3);
    padding: 15px;
}
```

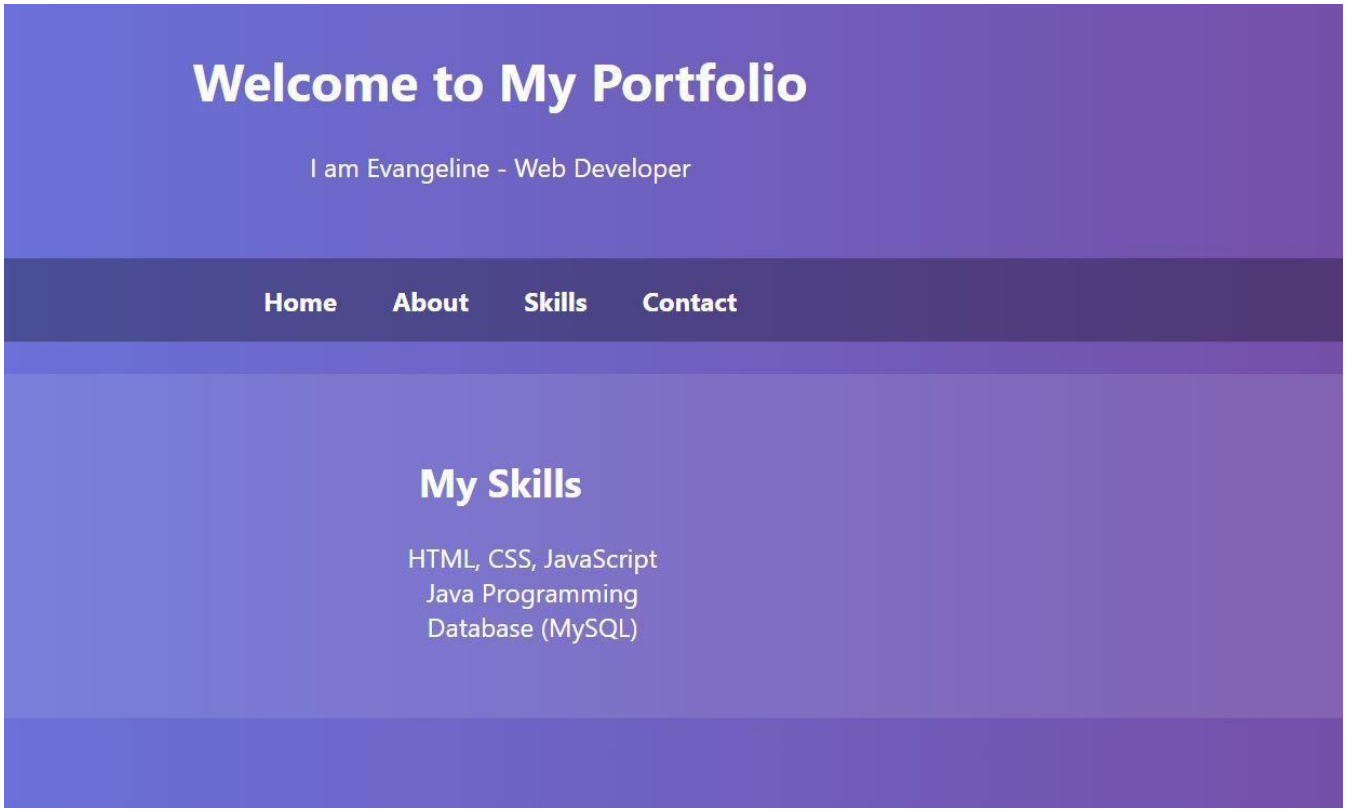
```
nav a {
    color: white;
    margin: 15px;
    cursor: pointer;
    font-weight: bold;
}

nav a:hover {
    color: #ffd700;
}

.section {
    display: none;
    background: rgba(255,255,255,0.1);
    margin: 20px;
    padding: 30px;
    border-radius: 15px;
}

button {
    padding: 10px 20px;
    background: #ffd700;
    border: none;
    color: black;
    cursor: pointer;
    border-radius: 5px;
}

button:hover {
    background: #ffcc00;
}

ul {
    list-style: none;
}

@media (max-width: 600px) {
    nav a {
        display: block;
        margin: 10px;
    }}
```

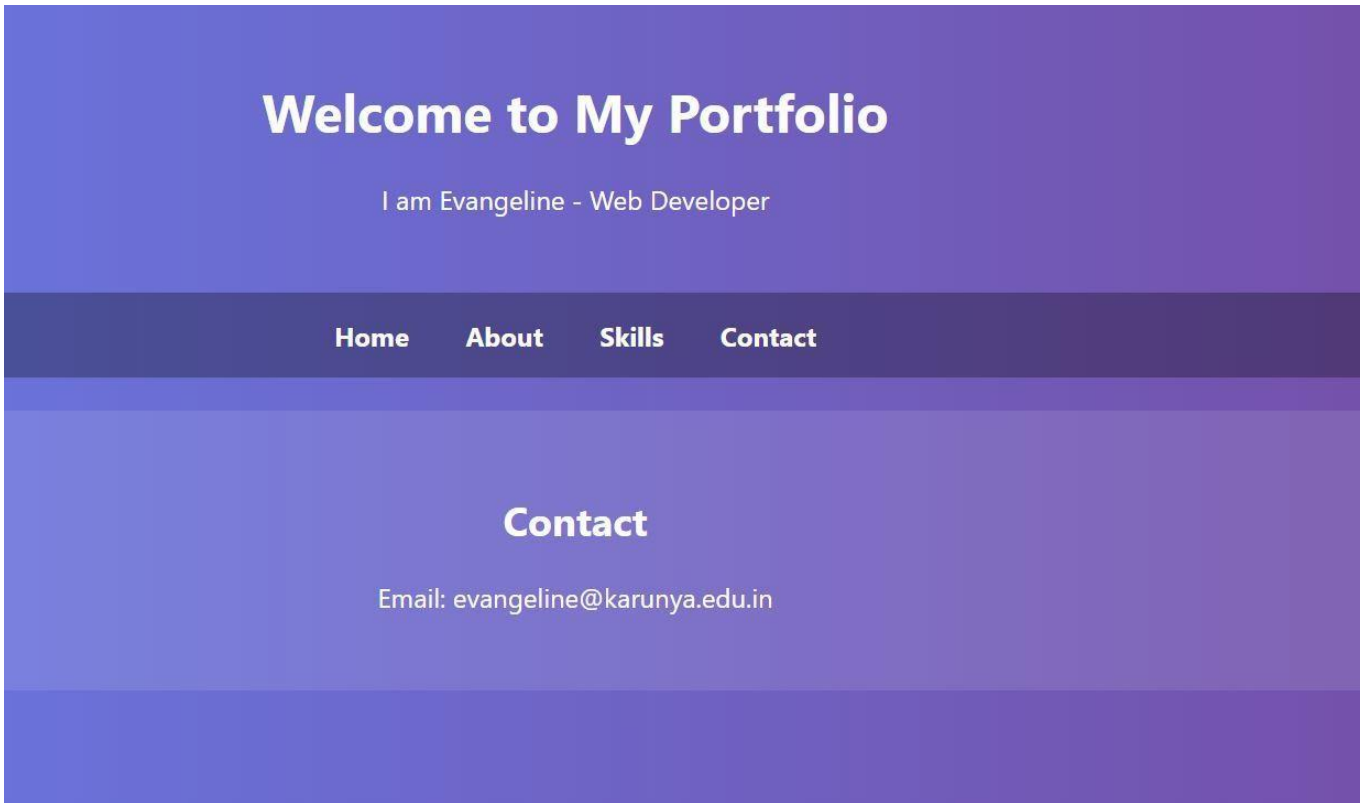
Input & Output






Welcome to My Portfolio
I am Evangeline - Web Developer
Home
About
Skills
Contact
About Me
I am a passionate student with strong interest in Web Development and Artificial Intelligence.
I enjoy building creative websites and exploring new technologies.
My goal is to become a skilled full-stack developer.


Welcome to My Portfolio
I am Evangeline - Web Developer
Home
About
Skills
Contact
My Skills
HTML, CSS, JavaScript
Java Programming
Database (MySQL)



Result

The responsive website was successfully designed and developed using AI tools, demonstrating effective layout generation, content creation, and dynamic functionality.